Capstone Project Part 5.1

```
!pip install requests python-dotenv
```

```
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (2.32.4)
Requirement already satisfied: python-dotenv in /usr/local/lib/python3.12/dist-packages (1.2.2)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests) (3.11)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests) (2.5
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests) (202
```

Part 1. Weather API Integration

```
# fd85d506c2ae4255b7694243240610

WEATHER_API_KEY = "xxx"

import requests

def get_weather(location):
    try:
        url = "http://api.weatherapi.com/v1/current.json"
        params = {
            "key": WEATHER_API_KEY,
            "q": location,
            "aqi": "no"
        }

        response = requests.get(url, params=params)
        response.raise_for_status()

        return response.json()

    except requests.exceptions.RequestException as e:
        raise Exception(f"Weather API error: {str(e)}")
```

```
# Test weather API integration
try:
    weather_data = get_weather("London")

    print(f"Weather in {weather_data['location']['name']}:")
    print(f"Temperature: {weather_data['current']['temp_c']}°C")
    print(f"Condition: {weather_data['current']['condition']['text']}")

except Exception as e:
    print(f"Error: {e}")
```

```
Weather in London:
Temperature: 8.0°C
Condition: Partly cloudy
```

Part 2. OpenAI/groq Function Calling Integration

```
!pip install groq
```

```
Collecting groq
  Downloading groq-1.1.2-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from groq) (4.13.0)
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.12/dist-packages (from groq) (1.9.0)
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from groq) (0.28.1)
Requirement already satisfied: pydantic<3,>=1.9.0 in /usr/local/lib/python3.12/dist-packages (from groq) (2.12.3)
Requirement already satisfied: sniffio in /usr/local/lib/python3.12/dist-packages (from groq) (1.3.1)
Requirement already satisfied: typing-extensions<5,>=4.14 in /usr/local/lib/python3.12/dist-packages (from groq)
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.12/dist-packages (from anyio<5,>=3.5.0->groq)
Requirement already satisfied: certifi in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->groq) (
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.12/dist-packages (from httpx<1,>=0.23.0->g
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.12/dist-packages (from httpcore==1.*->httpx<1,
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.12/dist-packages (from pydantic<3
Requirement already satisfied: pydantic-core==2.41.4 in /usr/local/lib/python3.12/dist-packages (from pydantic<3,
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python3.12/dist-packages (from pydantic
Downloading groq-1.1.2-py3-none-any.whl (141 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 141.7/141.7 kB 12.2 MB/s eta 0:00:00
Installing collected packages: groq
Successfully installed groq-1.1.2
```

```
from groq import Groq

client = Groq(api_key="xxx")



def extract_location(user_query):
    try:
        response = client.chat.completions.create(
            model="llama-3.1-8b-instant",
            messages=[
                {
                    "role": "system",
                    "content": "Extract only the city or location name from the user query. Return only the loca
                },
                {
                    "role": "user",
                    "content": user_query
                }
            ]
        )

        location = response.choices[0].message.content.strip()
        return location

    except Exception as e:
        raise Exception(f"Groq API error: {str(e)}")
```

```
try:
    query = "What's the weather like in Tokyo?"
    location = extract_location(query)

    print("Extracted Location:", location)

except Exception as e:
    print("Error:", e)
```

```
Extracted Location: Tokyo
```

```
def weather_agent(user_query):
    try:
        # Step 1: Extract location using LLM
        location = extract_location(user_query)

        # Step 2: Call Weather API
        weather_data = get_weather(location)

        # Step 3: Format response
        city = weather_data['location']['name']
        temp_c = weather_data['current']['temp_c']
        condition = weather_data['current']['condition']['text']

        return f"Current weather in {city}: {temp_c}°C, {condition}"

    except Exception as e:
        return f"Error: {str(e)}"
```

```
# Test queries
test_queries = [
    "What's the weather like in Singapore?",
    "Is it raining in London right now?",
    "Tell me the temperature in New York"
]

for query in test_queries:
    print(f"\nQuery: {query}")
    print(f"Response: {weather_agent(query)}")
```

```
Query: What's the weather like in Singapore?
Response: Current weather in Singapore: 30.1°C, Partly cloudy

Query: Is it raining in London right now?
Response: Current weather in London: 8.3°C, Overcast

Query: Tell me the temperature in New York
Response: Current weather in New York: 5.6°C, Mist
```

Extension Activities (Enhanced Implementation)

```
def weather_agent_enhanced(user_query):
    try:
        # Step 1: Extract location
        location = extract_location(user_query)

        if not location or len(location) < 2:
            return "Invalid query. Please provide a valid city name."

        # Step 2: Call Weather API
        weather_data = get_weather(location)

        # Step 3: Extract additional data
        city = weather_data['location']['name']
        country = weather_data['location']['country']
        local_time = weather_data['location']['localtime']

        temp_c = weather_data['current']['temp_c']
        temp_f = weather_data['current']['temp_f']
        feels_c = weather_data['current']['feelslike_c']
        humidity = weather_data['current']['humidity']
        wind_kph = weather_data['current']['wind_kph']
        wind_dir = weather_data['current']['wind_dir']
        condition = weather_data['current']['condition']['text']

        # Step 4: Add emoji
        condition_lower = condition.lower()
        if "rain" in condition_lower:
            emoji = "🌧️"
        elif "sun" in condition_lower or "clear" in condition_lower:
            emoji = "☀️"
        elif "cloud" in condition_lower:
            emoji = "☁️"
        else:
            emoji = "🌡️"

        # Step 5: Format response
        response = (
            f"{emoji} Weather in {city}, {country}\n"
            f"🕒 Time: {local_time}\n"
            f"🌡️ Temperature: {temp_c}°C / {temp_f}°F\n"
            f"🤔 Feels like: {feels_c}°C\n"
            f"💧 Humidity: {humidity}%\n"
            f"💨 Wind: {wind_kph} kph ({wind_dir})\n"
            f"📌 Condition: {condition}"
        )

        return response

    except Exception as e:
        error_msg = str(e).lower()

        if "no matching location" in error_msg:
            return "Invalid city name. Please try a valid location."

        elif "api" in error_msg:
            return "Weather service is currently unavailable. Please try again later."

        elif "timeout" in error_msg:
            return "Request timed out. Check your network connection."

        return f"Unexpected error: {str(e)}"
```

```
test_queries = [
    "Weather in Singapore",
    "Is it raining in London?",
    "Temperature in New York",
    "Weather in InvalidCity123"
]

for query in test_queries:
    print(f"\nQuery: {query}")
    print(weather_agent_enhanced(query))
```

```
Query: Weather in Singapore
☁️ Weather in Singapore, Singapore
```

```
🕒 Time: 2026-04-03 11:27
🌡️ Temperature: 31.1°C / 88.0°F
🤔 Feels like: 38.5°C
💧 Humidity: 59%
🌬️ Wind: 11.2 kph (ENE)
📌 Condition: Partly cloudy

Query: Is it raining in London?
🌧️ Weather in London, United Kingdom
🕒 Time: 2026-04-03 04:27
🌡️ Temperature: 8.1°C / 46.6°F
🤔 Feels like: 5.7°C
💧 Humidity: 87%
🌬️ Wind: 14.0 kph (SSW)
📌 Condition: Light rain

Query: Temperature in New York
🌧️ Weather in New York, United States of America
🕒 Time: 2026-04-02 23:27
🌡️ Temperature: 5.1°C / 41.2°F
🤔 Feels like: 1.2°C
💧 Humidity: 96%
🌬️ Wind: 19.8 kph (E)
📌 Condition: Light rain

Query: Weather in InvalidCity123
Weather service is currently unavailable. Please try again later.
```

```
## Testing Requirements Implementation


def run_tests():
    test_cases = {
        "Basic Queries": [
            "Weather in Singapore",
            "Temperature in London",
            "Is it raining in Tokyo?"
        ],
        "Error Cases": [
            "Weather in InvalidCity123",
            "",  # malformed query
            "asdfghjkl"  # random text
        ]
    }

    print("\n================ BASIC QUERY TESTS ================")
    for query in test_cases["Basic Queries"]:
        print(f"\nQuery: {query}")
        response = weather_agent_enhanced(query)
        print(response)

        # Validation checks
        assert "Temperature" in response or "°C" in response, "Temperature missing"
        assert "Condition" in response or "Weather" in response, "Condition missing"

    print("\n================ ERROR CASE TESTS ================")
    for query in test_cases["Error Cases"]:
        print(f"\nQuery: {query}")
        response = weather_agent_enhanced(query)
        print(response)

    print("\n================ ALL TESTS COMPLETED ================")


run_tests()
```

```
================ BASIC QUERY TESTS ================

Query: Weather in Singapore
☁️ Weather in Singapore, Singapore
🕒 Time: 2026-04-03 11:16
🌡️ Temperature: 30.1°C / 86.2°F
🤔 Feels like: 35.9°C
💧 Humidity: 66%
🌬️ Wind: 11.2 kph (ENE)
📌 Condition: Partly cloudy

Query: Temperature in London
🌡️ Weather in London, United Kingdom
🕒 Time: 2026-04-03 04:16
🌡️ Temperature: 8.3°C / 46.9°F
🤔 Feels like: 5.9°C
💧 Humidity: 81%
🌬️ Wind: 14.0 kph (SSW)
📌 Condition: Overcast
```

```
Query: Is it raining in Tokyo?
☁️ Weather in Tokyo, Japan
🕒 Time: 2026-04-03 12:31
🌡️ Temperature: 17.3°C / 63.1°F
🤔 Feels like: 17.3°C
💧 Humidity: 45%
🌬️ Wind: 16.6 kph (S)
📌 Condition: Partly Cloudy

================ ERROR CASE TESTS ================

Query: Weather in InvalidCity123
Weather service is currently unavailable. Please try again later.

Query:
☁️ Weather in One, Norway
🕒 Time: 2026-04-03 05:31
🌡️ Temperature: 0.2°C / 32.4°F
🤔 Feels like: -6.0°C
💧 Humidity: 80%
🌬️ Wind: 27.7 kph (SE)
📌 Condition: Cloudy

Query: asdfghjkl
☁️ Weather in New Found Out, Guyana
🕒 Time: 2026-04-02 23:31
🌡️ Temperature: 24.2°C / 75.6°F
🤔 Feels like: 27.1°C
💧 Humidity: 94%
🌬️ Wind: 4.0 kph (NE)
📌 Condition: Partly cloudy

================ ALL TESTS COMPLETED ================
```